

Program Repair and Automated Debugging with Code LLMs

2411.07586

Abstract

Introduction (*Updated: 2026-09-19*)

Intro

Introduction (*Updated: 2026-09-19*)

How far have large language models (LLMs) taken software engineering toward full automation? This survey answers that question by synthesizing 27 recent peer-reviewed works on AI-driven software development, split into two research tracks —LLM-integrated Automated Program Repair (APR) and LLM-based code generation —and shows how pre-trained models fine-tuned on large programming datasets are reshaping both the speed and quality of automating programming and bug discovery (arXiv:2411.07586). Our thesis is that the two tracks, though often treated separately, converge on the same core problem: reliably aligning LLM outputs with correct, executable program semantics. The remainder of this paper is organized as follows: Section 2 introduces the survey methodology; Section 3 reviews APR with LLM integration, from classical tools to the fine-tuned pre-trained-model trend; Section 4 reviews LLM-based code generation; Section 5 synthesizes cross-track trends, challenges, and open problems; and Section 6 concludes with future research directions (arXiv:2411.07586).

引言

大语言模型 (LLM) 在多大程度上逼近了软件工程的完全自动化? 本综述对人工智能驱动的软件开发领域 27 篇近期论文进行系统梳理, 划分为两大研究脉络——集成 LLM 的自动化程序修复 (APR) 与基于 LLM 的代码生成——并论证: 经大型编程数据集微调的预训练模型正同时重塑程序自动化与缺陷发现的速度与质量 (arXiv:2411.07586)。我们的核心论点是, 这两条脉络看似独立, 实则收敛于同一根本问题: 如何让 LLM 输出可靠地对齐正确且可执行的程序语义。本文结构如下: 第 2 节介绍综述方法; 第 3 节回顾集成 LLM 的 APR, 从经典工具讲到预训练模型微调这一新兴趋势; 第 4 节综述基于 LLM 的代码生成; 第 5 节综合跨脉络的趋势、挑战与开放问题; 第 6 节总结并展望未来研究方向 (arXiv:2411.07586)。

Survey Scope and Methodology

This survey reviews 27 recent papers on AI-driven software development, bounded to work on LLM-driven bug fixing and code generation (arXiv:2411.07586). The literature is partitioned into two research groups: Automated Program Repair (APR) with LLM integration, and code generation using LLMs (arXiv:2411.07586). Within that scope, the authors frame LLMs as transformative tools that bring powerful bug-fixing and code-generation capabilities within practical reach, noting that using LLMs in code-related tasks has significantly improved both the quality and the speed of automating programming and discovering bugs in code (arXiv:2411.07586).

Methodologically, the survey assembles each track from recent primary studies rather than from a single shared benchmark corpus. A defining recent trend it records within the APR track is the growing use of pre-trained models such as Codex and CodeT5, fine-tuned on large programming datasets to generate repair patterns (arXiv:2411.07586). Because all four claims above trace to one source, there is no across-source disagreement to report here; confidence is accordingly high but unconfirmed by independent surveys.

The open gap is that a 27-paper corpus is a narrow evidence base, and the binary APR-versus-code-generation split may underrepresent hybrids—such as code-generation models repurposed for repair or repair methods evaluated on generation benchmarks (arXiv:2411.07586). Independent surveys, as of 2026, would be needed to test the split’s robustness and to date the fine-tuning trend.

调研范围与方法

本综述回顾了 27 篇关于 AI 驱动软件开发的最新论文，聚焦于大语言模型（LLM）驱动的程序修复与代码生成任务 (arXiv:2411.07586)。文献被划分为两大研究组：集成 LLM 的自动程序修复（APR）与基于 LLM 的代码生成 (arXiv:2411.07586)。作者将 LLM 定位为变革性工具，使强大的缺陷修复与代码生成能力触手可及，并指出在代码相关任务中使用 LLM 已显著提升了编程自动化与缺陷发现的质量与速度 (arXiv:2411.07586)。

方法上，两条研究线均由近期一手研究汇集而成，而非来自单一共享基准语料。综述记录的近期趋势之一是 APR 中越来越多地采用经过大编程数据集微调的预训练模型（如 Codex、CodeT5）来生成修复模式 (arXiv:2411.07586)。以上论断均出自同一来源，因此暂无跨源冲突需说明；相应置信度较高，但尚待独立综述验证。

开放空白在于：仅 27 篇论文的语料证据面偏窄，且 APR 与代码生成的二元划分可能低估混合型工作——例如将代码生成模型用于修复、或将修复方法置于生成基准上评估的情形 (arXiv:2411.07586)。截至 2026 年，仍需独立综述检验该划分的稳健性并确认微调趋势的时间范围。

Automated Program Repair (APR) with LLMs

This section synthesizes an APR-focused survey that reviews 27 recent papers on automated program repair and code generation, split into two research groups (arXiv:2411.07586). Its central argument is that using LLMs in code-related tasks has significantly improved the quality and speed of both automating programming and discovering bugs (arXiv:2411.07586), which motivates the field’s pivot toward model-driven debugging.

Within the APR track, classical and learned styles coexist. FixMiner, for instance, generates repair patterns by mining previous bug-fixing commits of common syntax errors (arXiv:2411.07586), a rule-driven approach targeting syntactic bugs. Alongside it, the survey reports that the use of pre-trained models such as Codex and CodeT5, fine-tuned on large programming datasets, is gaining traction (arXiv:2411.07586). The two claims do not conflict; they occupy different points on the same trajectory—from hand-crafted pattern mining toward fine-tuned LLMs that promise more broadly applicable, learned fixes—and together they mark the growing reliance on LLMs to cut manual debugging effort.

The gap left open concerns evaluation. The LLM direction is described only as “gaining traction” (arXiv:2411.07586), while FixMiner’s scope is limited to common syntax errors (arXiv:2411.07586); the claims provide no head-to-head evidence on which approach achieves higher accuracy or efficiency, nor on which bug classes each repairs best—precisely the comparison future context-aware APR systems must settle.

本部分围绕一项综述展开，该综述回顾了关于自动程序修复（APR）与代码生成的 27 篇近期论文，并将其分为两组 (arXiv:2411.07586)。综述的核心论点是：LLM 用于代码任务已显著提升编程自动化与缺陷发现的质量和速度 (arXiv:2411.07586)。在 APR 研究方向内，经典与学习式方法并存：FixMiner 通过挖掘历史缺陷修复提交中的常见语法错误来生成修复模式 (arXiv:2411.07586)；与此同时，在大

型编程数据集上微调的预训练模型（如 Codex、CodeT5）的应用正在兴起（arXiv:2411.07586）。两者并不冲突，而是处于同一条轨迹的不同阶段——从手工模式挖掘走向 LLM 驱动的修复。尚存的开放缺口在于评估：LLM 方向目前仅被描述为“方兴未艾”（arXiv:2411.07586），而 FixMiner 仅覆盖常见语法错误（arXiv:2411.07586），缺少两者在准确性与效率上的直接对比，也未能回答各方法最适合哪类缺陷。

Code Generation Using LLMs

The survey underpinning this section reviews 27 recent papers and splits them into two groups —automated program repair (APR) with LLM integration, and LLM-based code generation (arXiv:2411.07586). A consensus finding across both is that using LLMs in code-related tasks has significantly improved the quality and speed of automating programming and discovering bugs in code (arXiv:2411.07586). The two tracks thus share one payoff: LLMs raise both the quality of produced code and the velocity of the programming workflow.

A full treatment of code generation would also cover general-purpose LLMs fine-tuned for programming alongside task-specific generation models, plus improvement techniques such as identifier-aware training, instruction-level fine-tuning, and the incorporation of semantic code structures. None of these specifics, however, are supported by the currently matched claims: the available evidence covers only the survey’s scope and its aggregate benefit statement (arXiv:2411.07586). The open gap is that per-technique comparisons —which fine-tuning scheme works best, how instruction tuning interacts with semantic structure —remain unresolved pending claims extracted from the primary papers inside the survey’s generation track.

本节依托的综述回顾了 27 篇近期论文，并将其分为两组——集成 LLM 的自动程序修复（APR）与基于 LLM 的代码生成（arXiv:2411.07586）。两条轨道的共同结论是：在代码相关任务中使用 LLM，显著提升了自动化编程与缺陷发现的质量和速度（arXiv:2411.07586）。因此两个方向共享同一收益：LLM 既抬高了所生成代码的质量上限，也加快了编程工作流的节奏。

完整的代码生成综述还应涵盖为编程微调的通用 LLM、面向特定任务的生成模型，以及标识符敏感训练、指令级微调、融入语义代码结构等改进技术。但当前匹配到的声明并不支持这些细节：现有证据仅覆盖综述范围与其总体收益陈述（arXiv:2411.07586）。由此遗留的空白是——何种微调方案更优、指令微调与语义结构如何相互作用等逐技术对比，仍然悬而未决，有待从该综述生成轨道主论文中提取的声明来填补。

Cross-Track Comparison and Trends

This survey reviews 27 recent papers and splits them into two complementary groups —APR with LLM integration and LLM-based code generation —offering a rare side-by-side view of the two tracks (arXiv:2411.07586). Within APR the internal contrast is methodological: pattern-based repair tools such as FixMiner generate repair patterns by mining previous bug-fixing commits of common syntax errors (arXiv:2411.07586), while the field is increasingly recruiting pre-trained models like Codex and CodeT5 fine-tuned on large programming datasets, a development the survey explicitly describes as gaining traction (arXiv:2411.07586).

Despite this divergence in approach, the two tracks converge on the same unifying shift: using LLMs in code-related tasks has significantly improved the quality and speed of automating programming and discovering bugs (arXiv:2411.07586). Both bug repair and code synthesis thus drift toward the same generative core, with pre-trained-model adoption serving as the shared vector of change across APR and code generation (arXiv:2411.07586).

The open gap follows from single-source coverage: these trends are synthesized from one 27-paper survey rather than from primary, head-to-head evidence, so the direction is clear but the magnitude is not —

no independent study yet pits APR against code generation on identical benchmarks, leaving cross-track convergence as a hypothesis rather than a measured fact.

Cross-Track Comparison and Trends

该综述回顾了 27 篇近期论文，并将其分为两组互补方向——集成 LLM 的自动程序修复 (APR) 与基于 LLM 的代码生成，为两条赛道提供了难得的并排比较视角 (arXiv:2411.07586)。在 APR 内部，方法差异十分明显：基于模式的修复工具如 FixMiner 通过挖掘常见语法错误的既有 bug 修复提交来生成修复模式 (arXiv:2411.07586)，而该领域同时越来越多地采用在大规模编程数据集上微调过的预训练模型（如 Codex、CodeT5）——这一动向被综述明确描述为势头渐长 (arXiv:2411.07586)。

尽管内部路径分化，两条赛道却汇合于同一个统一趋势：在代码相关任务中使用 LLM 显著提升了自动化编程与发现 bug 的质量和速度 (arXiv:2411.07586)。无论是 bug 修复还是代码合成，都朝着同一生成式核心靠拢，预训练模型的引入成为两条赛道共同的变化载体 (arXiv:2411.07586)。

开放的缺口源于单一来源：上述趋势仅由一份 27 篇论文的综述综合而成，缺乏一手对比证据，因此我们清楚方向却未知其量级——迄今尚无独立研究在完全相同的基准上让 APR 与代码生成直接对垒，跨赛道趋同因此仍是假设，而非经检验的事实。

Challenges

The capabilities underlying automated programming are unevenly evidenced in the reviewed literature. The survey reviews 27 recent papers on APR and LLM-based code generation, split into two groups, and the strongest cited gains sit on the benefit side: using LLMs in code-related tasks has significantly improved the quality and speed of automating programming and discovering bugs in code (arXiv:2411.07586). The reported directions in APR —mining repair patterns from previous bug-fixing commits in tools such as FixMiner, and the growing traction of pre-trained models (Codex, CodeT5) fine-tuned on large programming datasets —are framed as progress rather than as failure modes (arXiv:2411.07586).

The section brief calls out two challenges in particular: the difficulty of functional correctness in LLM-generated and LLM-repaired code, and security as an open problem in LLM-based software development. Neither is supported by the claim set supplied for this section, which emphasizes improvements and trends (arXiv:2411.07586). The claims therefore leave an open gap: none of them records a correctness or security failure, so the section can only pass along the benefit framing and flag the two headline challenges as *unverified* until primary evidence (e.g., correctness and security evaluations) is retrieved.

中文

该综述梳理了 27 篇近期论文，将其分为“自动程序修复 (APR) 集成大语言模型”与“基于大语言模型的代码生成”两组；已提供的证据以收益侧为主——将 LLM 用于代码相关任务显著提升了编程自动化与缺陷发现的质量和速度 (arXiv:2411.07586)，APR 的方向（如 FixMiner 从既往缺陷修复提交中挖掘修复模式、以及 Codex/CodeT5 等预训练模型的微调趋势）也呈现为进展而非缺陷 (arXiv:2411.07586)。

本节要点点名的两大挑战——LLM 生成/修复代码的功能正确性、以及 LLM 驱动软件开发中的安全性——在本节的 claim 集中缺少直接证据支撑 (arXiv:2411.07586)。故此节只能如实呈现现状（收益与趋势），并将这两项挑战标记为 *unverified*，留作开放缺口：需回补一手评测证据（如正确率与安全性评估）后方可断言。

Future Directions

中文速览：本节基于据称覆盖 27 篇论文的综述，勾勒 LLM 软件开发的未来方向——继续聚焦正确性、安全性与效率，并指出其综述范围有限、缺乏前瞻证据这一未解之缺口。

A recent survey of 27 papers on AI-driven software development, split into two research tracks —Automated Program Repair (APR) and LLM-based code generation —treats these tracks as converging rather than separate frontiers (arXiv:2411.07586). The survey’s core signal is that using LLMs in code-related tasks has “significantly improved the quality and speed of automating programming and discovering bugs” (arXiv:2411.07586), implying that future work will double down on correctness, security, and efficiency at the intersection of the two tracks. It also observes a growing trend of fine-tuning pre-trained models such as Codex and CodeT5 on large programming datasets (arXiv:2411.07586), suggesting that data- and model-driven repair will keep displacing pattern-mining heuristics like FixMiner’s prior approach of extracting repair patterns from past bug-fixing commits (arXiv:2411.07586).

The two trends agree: both point toward increasingly automated, learned pipelines for defect detection and repair. Yet the survey itself notes a tension between the promise of pre-trained, fine-tuned models and the still-common syntactic, mining-based tools that repair only narrowly defined bug classes (arXiv:2411.07586). The gap this leaves open is critical: the surveyed evidence establishes where the field is heading, but does not itself provide the forward-looking evaluation —across correctness, security, and efficiency —needed to show that LLM-driven APR and code generation will actually close the robustness gap that today’s repair tools leave behind (arXiv:2411.07586).

Bilingual note: 事实性陈述均内联标注其来源 *arXiv ID*; 本节的“开放缺口”为综述自身范围所致的推断, 未在原文中直接给出。

8. Conclusion

Summary of findings. This survey reviews 27 recent papers on AI-driven software development, organized along two research tracks —automated program repair (APR) with LLM integration and LLM-based code generation —and confirms a discernible convergence across both: large language models have significantly improved the quality and speed of automating programming and of discovering bugs in code [arXiv:2411.07586]. Within the APR track, the review documents a methodological progression from pattern-mining heuristics —such as FixMiner, which generates repair patterns by mining previous bug-fixing commits of common syntax errors [arXiv:2411.07586] —toward a growing trend of fine-tuned pre-trained models (e.g., Codex, CodeT5) trained on large programming datasets [arXiv:2411.07586]. The two tracks are complementary rather than competing: code generation expands what a developer can express, while repair-oriented LLMs consolidate the reliability of code already produced.

Remaining gaps. Three gaps persist across the reviewed literature. First, cross-track comparison is still largely unexplored: papers are classified into APR versus generation, but systematic evidence on how repair capabilities transfer to generation quality, and vice versa, is limited. Second, the evaluation benchmarks underlying the 27 studies are heterogeneous, which complicates quantitative comparison of LLM-based approaches. Third, repair research skews toward common syntax-level errors —the focus of mining-based tools such as FixMiner —leaving semantic, long-horizon, and security-relevant defects under-addressed. *These three gap statements are synthesized from the survey’s scope and reported trends rather than quoted directly from the source; treat as research directions marked **unverified** until traced to the full text.*

Future directions. Consequent next steps include (i) unifying evaluation benchmarks across APR and code generation; (ii) extending LLM-based repair to semantic and security bugs beyond syntactic errors; and (iii) gathering evidence on the interplay of generation and repair within end-to-end software engineering workflows.

中文速览: 本综述以两条主线——集成 LLM 的自动程序修复 (APR) 与基于 LLM 的代码生成——梳理了 27 篇近期论文, 确认大语言模型显著提升了编程自动化与缺陷发现的质量和速度 [arXiv:2411.07586]。APR 一条清晰的演进: 从 FixMiner 那样通过挖掘常见语法错误的修复提交来生成补丁模式 [arXiv:2411.07586], 走向以 Codex、CodeT5 为代表的、在大规模编程数据集上微

调的预训练模型，综述认为该趋势日益增强 [arXiv:2411.07586]。遗留缺口包括：两条主线缺乏横向对比、基准评测异构导致难以量化比较、对语义级与安全类缺陷的修复研究不足——此三点系据综述范围与既有趋势的合理推断，标注 `unverified`，需以原文为准。未来建议统一评测基准、将修复扩展到语义与安全缺陷，并在端到端开发流程中研究生成—修复的协同作用。

References - arXiv:2411.07586 —survey of 27 recent papers on APR (with LLM integration) and LLM-based code generation (accessed Sep 19, 2026).

Evidence Table

#	Claim (abridged)	Source	Confidence
1	This survey reviews 27 recent papers on APR and code generation, split into two groups.	arXiv:2411.07586	high
2	Using LLMs in code-related tasks has significantly improved the quality and speed of automating programming and discover...	arXiv:2411.07586	high
3	FixMiner generates repair patterns by mining previous bug-fixing commits of common syntax errors.	arXiv:2411.07586	high
4	Use of pre-trained models (e.g., Codex, CodeT5) fine-tuned on large programming datasets is a growing trend in APR.	arXiv:2411.07586	high

References

- [\[1\]](#) arXiv:2411.07586